

```
CREATE DATABASE database_name;  
USE database_name;
```

```
CREATE DATABASE EventManagement;  
USE EventManagement;
```

INT → Integer
VARCHAR(n) → Text
TEXT → Large text
DATE → Date only
DATETIME → Date + Time
ENUM → Fixed values

PRIMARY KEY → Unique identifier
AUTO_INCREMENT → Auto numbering
NOT NULL → Cannot be empty
UNIQUE → No duplicates
FOREIGN KEY → Relationship
CHECK → Validation rule

COUNT() → Count
AVG() → Average
SUM() → Total
MIN() → Minimum
MAX() → Maximum

INNER JOIN → Matching rows
LEFT JOIN → All left rows

GROUP BY → Group records
HAVING → Filter groups
ORDER BY → Sort
LIMIT → Restrict rows

```
CREATE TABLE table_name(  
    var_name var_type additional,  
    FOREIGN KEY (var1)  
        REFERENCES other_table(var2)  
);
```

```
INSERT INTO table_name  
(var1,var2)  
VALUES  
(val11',val12'),  
(val21',val22');
```

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,
```

```

full_name VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
city VARCHAR(100) NOT NULL,
registration_date DATE NOT NULL
);

INSERT INTO Users
(full_name,email,city,registration_date)
VALUES
('Alice Johnson','alice@example.com','New York','2024-12-01'),
('Bob Smith','bob@example.com','Los Angeles','2024-12-05'),
('Charlie Lee','charlie@example.com','Chicago','2024-12-10'),
('Diana King','diana@example.com','New York','2025-01-15'),
('Ethan Hunt','ethan@example.com','Los Angeles','2025-02-01');

CREATE TABLE Events (
    event_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(200) NOT NULL,
    description TEXT,
    city VARCHAR(100) NOT NULL,
    start_date DATETIME NOT NULL,
    end_date DATETIME NOT NULL,
    status ENUM('upcoming','completed','cancelled'),
    organizer_id INT,
    FOREIGN KEY (organizer_id)
        REFERENCES Users(user_id)
);

INSERT INTO Events
(title,description,city,start_date,end_date,status,organizer_id)
VALUES
(
'Tech Innovators Meetup',
'A meetup for tech enthusiasts.',
'New York',
'2025-06-10 10:00:00',
'2025-06-10 16:00:00',
'upcoming',
1
),
(
'AI & ML Conference',
'Conference on AI and ML advancements.',
'Chicago',
'2025-05-15 09:00:00',
'2025-05-15 17:00:00',
'completed',
3
),

```

```
(  
'Frontend Development Bootcamp',  
'Hands-on training on frontend tech.',  
'Los Angeles',  
'2025-07-01 10:00:00',  
'2025-07-03 16:00:00',  
'upcoming',  
2  
);
```

```
CREATE TABLE Sessions (  
    session_id INT PRIMARY KEY AUTO_INCREMENT,  
    event_id INT,  
    title VARCHAR(200) NOT NULL,  
    speaker_name VARCHAR(100) NOT NULL,  
    start_time DATETIME NOT NULL,  
    end_time DATETIME NOT NULL,  
    FOREIGN KEY (event_id)  
        REFERENCES Events(event_id)  
);
```

```
INSERT INTO Events  
(title,description,city,start_date,end_date,status,organizer_id)  
VALUES  
(  
'Tech Innovators Meetup',  
'A meetup for tech enthusiasts.',  
'New York',  
'2025-06-10 10:00:00',  
'2025-06-10 16:00:00',  
'upcoming',  
1  
)  
(  
'AI & ML Conference',  
'Conference on AI and ML advancements.',  
'Chicago',  
'2025-05-15 09:00:00',  
'2025-05-15 17:00:00',  
'completed',  
3  
)  
(  
'Frontend Development Bootcamp',  
'Hands-on training on frontend tech.',  
'Los Angeles',  
'2025-07-01 10:00:00',  
'2025-07-03 16:00:00',  
'upcoming',
```

```
2  
);
```

```
CREATE TABLE Registrations (  
    registration_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    event_id INT,  
    registration_date DATE NOT NULL,  
    FOREIGN KEY (user_id)  
        REFERENCES Users(user_id),  
    FOREIGN KEY (event_id)  
        REFERENCES Events(event_id)  
);
```

```
INSERT INTO Registrations  
(user_id,event_id,registration_date)  
VALUES  
(1,1,'2025-05-01'),  
(2,1,'2025-05-02'),  
(3,2,'2025-04-30'),  
(4,2,'2025-04-28'),  
(5,3,'2025-06-15');
```

```
CREATE TABLE Feedback (  
    feedback_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    event_id INT,  
    rating INT CHECK (rating BETWEEN 1 AND 5),  
    comments TEXT,  
    feedback_date DATE NOT NULL,  
    FOREIGN KEY (user_id)  
        REFERENCES Users(user_id),  
    FOREIGN KEY (event_id)  
        REFERENCES Events(event_id)  
);
```

```
INSERT INTO Feedback  
(user_id,event_id,rating,comments,feedback_date)  
VALUES  
(  
    3,  
    2,  
    4,  
    'Great insights!',  
    '2025-05-16'  
),  
(  
    4,
```

```
2,
5,
'Very informative.',
'2025-05-16'
),
(
2,
1,
3,
'Could be better.',
'2025-06-11'
);
```

```
CREATE TABLE Resources (
  resource_id INT PRIMARY KEY AUTO_INCREMENT,
  event_id INT,
  resource_type ENUM('pdf','image','link'),
  resource_url VARCHAR(255) NOT NULL,
  uploaded_at DATETIME NOT NULL,
  FOREIGN KEY (event_id)
    REFERENCES Events(event_id)
);
```

```
INSERT INTO Resources
(event_id,resource_type,resource_url,uploaded_at)
VALUES
(
1,
'pdf',
'https://portal.com/resources/tech_meetup_agenda.pdf',
'2025-05-01 10:00:00'
),
(
2,
'image',
'https://portal.com/resources/ai_poster.jpg',
'2025-04-20 09:00:00'
),
(
3,
'link',
'https://portal.com/resources/html5_docs',
'2025-06-25 15:00:00'
);
```

1. User Upcoming Events

Show a list of all upcoming events a user is registered for in their city, sorted by date.

```

SELECT u.full_name,
       e.title,
       e.city,
       e.start_date
FROM Users u
JOIN Registrations r ON u.user_id = r.user_id
JOIN Events e ON r.event_id = e.event_id
WHERE e.status = 'upcoming'
      AND u.city = e.city
ORDER BY e.start_date;

```

2. Top Rated Events

Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions

```

SELECT e.event_id,
       e.title,
       AVG(f.rating) AS avg_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(f.feedback_id) >= 10
      AND AVG(f.rating) = (
        SELECT MAX(avg_rating)
        FROM (
          SELECT AVG(rating) AS avg_rating
          FROM Feedback
          GROUP BY event_id
          HAVING COUNT(feedback_id) >= 10
        ) t
      );

```

3. Inactive Users

Retrieve users who have not registered for any events in the last 90 days.

```

SELECT u.user_id,
       u.full_name
FROM Users u
LEFT JOIN Registrations r
      ON u.user_id = r.user_id
      AND r.registration_date >= DATE_SUB(CURDATE(), INTERVAL 90 DAY)
WHERE r.registration_id IS NULL;

```

4. Peak Session Hours

Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```

SELECT e.title,
       COUNT(s.session_id) AS session_count
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id

```

```
WHERE HOUR(s.start_time) BETWEEN 10 AND 11
GROUP BY e.event_id, e.title;
```

5. Most Active Cities

List the top 5 cities with the highest number of distinct user registrations.

```
SELECT e.city,
       COUNT(DISTINCT r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.city
ORDER BY total_registrations DESC
LIMIT 5;
```

6. Event Resource Summary

Generate a report showing the number of resources (PDFs, images, links) uploaded for each Event.

```
SELECT e.title,
       SUM(CASE WHEN resource_type='pdf' THEN 1 ELSE 0 END) AS pdf_count,
       SUM(CASE WHEN resource_type='image' THEN 1 ELSE 0 END) AS image_count,
       SUM(CASE WHEN resource_type='link' THEN 1 ELSE 0 END) AS link_count
FROM Events e
LEFT JOIN Resources r
      ON e.event_id = r.event_id
GROUP BY e.event_id, e.title;
```

7. Low Feedback Alerts

List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```
SELECT u.full_name,
       e.title,
       f.rating,
       f.comments
FROM Feedback f
JOIN Users u
      ON f.user_id = u.user_id
JOIN Events e
      ON f.event_id = e.event_id
WHERE f.rating < 3;
```

8. Sessions per Upcoming Event

Display all upcoming events with the count of sessions scheduled for them.

```
SELECT e.title,
       COUNT(s.session_id) AS total_sessions
FROM Events e
LEFT JOIN Sessions s
      ON e.event_id = s.event_id
```

```
WHERE e.status = 'upcoming'  
GROUP BY e.event_id, e.title;
```

9. Organizer Event Summary

For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```
SELECT u.full_name,  
       e.status,  
       COUNT(*) AS total_events  
FROM Users u  
JOIN Events e  
      ON u.user_id = e.organizer_id  
GROUP BY u.full_name, e.status  
ORDER BY u.full_name;
```

10. Feedback Gap

Identify events that had registrations but received no feedback at all.

```
SELECT DISTINCT e.event_id,  
               e.title  
FROM Events e  
JOIN Registrations r  
      ON e.event_id = r.event_id  
LEFT JOIN Feedback f  
      ON e.event_id = f.event_id  
WHERE f.feedback_id IS NULL;
```

11. Daily New User Count

Find the number of users who registered each day in the last 7 days.

```
SELECT registration_date,  
       COUNT(*) AS new_users  
FROM Users  
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 7 DAY)  
GROUP BY registration_date  
ORDER BY registration_date;
```

12. Event with Maximum Sessions

List the event(s) with the highest number of sessions.

```
SELECT e.event_id,  
       e.title,  
       COUNT(s.session_id) AS total_sessions  
FROM Events e  
JOIN Sessions s ON e.event_id = s.event_id  
GROUP BY e.event_id, e.title  
HAVING COUNT(s.session_id) = (  
    SELECT MAX(session_count)  
    FROM (
```



```
        SELECT COUNT(*) AS session_count
        FROM Sessions
        GROUP BY event_id
    ) t
);
```

13. Average Rating per City

Calculate the average feedback rating of events conducted in each city.

```
SELECT e.city,
       ROUND(AVG(f.rating),2) AS average_rating
FROM Events e
JOIN Feedback f ON e.event_id = f.event_id
GROUP BY e.city;
```

14. Most Registered Events

List top 3 events based on the total number of user registrations.

```
SELECT e.title,
       COUNT(r.registration_id) AS total_registrations
FROM Events e
JOIN Registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_registrations DESC
LIMIT 3;
```

15. Event Session Time Conflict

Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

```
SELECT s1.event_id,
       s1.title AS session1,
       s2.title AS session2
FROM Sessions s1
JOIN Sessions s2
    ON s1.event_id = s2.event_id
   AND s1.session_id < s2.session_id
   AND s1.start_time < s2.end_time
   AND s1.end_time > s2.start_time;
```

16. Unregistered Active Users

Find users who created an account in the last 30 days but haven't registered for any events.

```
SELECT u.user_id,
       u.full_name
FROM Users u
LEFT JOIN Registrations r
    ON u.user_id = r.user_id
WHERE u.registration_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
AND r.registration_id IS NULL;
```

17. Multi-Session Speakers

Identify speakers who are handling more than one session across all events.

```
SELECT speaker_name,  
       COUNT(*) AS total_sessions  
FROM Sessions  
GROUP BY speaker_name  
HAVING COUNT(*) > 1;
```

18. Resource Availability Check

List all events that do not have any resources uploaded.

```
SELECT e.event_id,  
       e.title  
FROM Events e  
LEFT JOIN Resources r  
      ON e.event_id = r.event_id  
WHERE r.resource_id IS NULL;
```

19. Completed Events with Feedback Summary

For completed events, show total registrations and average feedback rating.

```
SELECT e.title,  
       COUNT(DISTINCT r.registration_id) AS total_registrations,  
       ROUND(AVG(f.rating),2) AS average_rating  
FROM Events e  
LEFT JOIN Registrations r  
      ON e.event_id = r.event_id  
LEFT JOIN Feedback f  
      ON e.event_id = f.event_id  
WHERE e.status = 'completed'  
GROUP BY e.event_id, e.title;
```

20. User Engagement Index

For each user, calculate how many events they attended and how many feedbacks they submitted.

```
SELECT u.full_name,  
       COUNT(DISTINCT r.event_id) AS events_attended,  
       COUNT(DISTINCT f.feedback_id) AS feedbacks_submitted  
FROM Users u  
LEFT JOIN Registrations r  
      ON u.user_id = r.user_id  
LEFT JOIN Feedback f  
      ON u.user_id = f.user_id  
GROUP BY u.user_id, u.full_name;
```

21. Top Feedback Providers

List top 5 users who have submitted the most feedback entries.

```

SELECT u.full_name,
       COUNT(f.feedback_id) AS feedback_count
FROM Users u
JOIN Feedback f
     ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY feedback_count DESC
LIMIT 5;

```

22. Duplicate Registrations Check

Detect if a user has been registered more than once for the same event.

```

SELECT user_id,
       event_id,
       COUNT(*) AS duplicate_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1;

```

23. Registration Trends

Show a month-wise registration count trend over the past 12 months.

```

SELECT DATE_FORMAT(registration_date,'%Y-%m') AS month,
       COUNT(*) AS total_registrations
FROM Registrations
WHERE registration_date >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY DATE_FORMAT(registration_date,'%Y-%m')
ORDER BY month;

```

24. Average Session Duration per Event

Compute the average duration (in minutes) of sessions in each event.

```

SELECT e.title,
       ROUND(
         AVG(
           TIMESTAMPDIFF(
             MINUTE,
             s.start_time,
             s.end_time
           )
         ),2
       ) AS avg_duration_minutes
FROM Events e
JOIN Sessions s
     ON e.event_id = s.event_id
GROUP BY e.event_id, e.title;

```

25. Events Without Sessions

List all events that currently have no sessions scheduled under them.

```
SELECT e.event_id,  
       e.title  
FROM Events e  
LEFT JOIN Sessions s  
      ON e.event_id = s.event_id  
WHERE s.session_id IS NULL;
```